

PRACTICAL WEEK 12:

1. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Code:

```
1 10W PICO 5.09 File: mk.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <pthread.h>
5 #include <semaphore.h>
6 #include <unistd.h>
7 #include <time.h>
8
9 #define ITEMS 100000
10
11 int *buffer;
12 int buffer_size;
13 int in = 0;
14 int out = 0;
15
16 sem_t empty; // Counts empty slots
17 sem_t full; // Counts filled slots
18 pthread_mutex_t mutex;
19
20 void *producer(void *arg)
21 {
22     for (int i = 0; i < ITEMS; i++)
23     {
24         // Wait for an empty slot
25         sem_wait(&empty);
26
27         // Enter critical section
28         pthread_mutex_lock(&mutex);
29
30         buffer[in] = i;
31         in = (in + 1) % buffer_size;
32
33         pthread_mutex_unlock(&mutex);
34
35         // Increase number of full slots
36         sem_post(&full);
37     }
38     return NULL;
39 }
40
41 void *consumer(void *arg)
42 {
43     long long sum = 0;
44
45     for (int i = 0; i < ITEMS; i++)
46     {
47         // Wait for an available item
48         sem_wait(&full);
49
50         // Enter critical section
51         pthread_mutex_lock(&mutex);
52
53         int item = buffer[out];
54         out = (out + 1) % buffer_size;
55
56         pthread_mutex_unlock(&mutex);
57
58         // Increase number of empty slots
59         sem_post(&empty);
60
61         sum += item;
62     }
63
64     printf("Consumer checksum: %lld\n", sum);
65
66     return NULL;
67 }
68
69 Get Help WriteOut Read File Prev Pg Out Text Cur Pos
70 Exit Justify Where is Next Pg UnOut Text To Spell
```

Output:

```
1 screen@killeddy@rees-MacBook-Pro ~ % nano mk.c
2 screen@killeddy@rees-MacBook-Pro ~ % gcc mk.c -o mk
3 mk.c:86:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
4 86 |     sem_init(&empty, 0, buffer_size);
5    |     ^
6 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
7 55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
8    |
9 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:214:40: note: expanded from macro '__deprecated'
10 214 | #define __deprecated __attribute__((__deprecated__))
11    |
12 mk.c:87:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
13 87 |     sem_init(&full, 0, 0);
14    |     ^
15 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
16 55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
17    |
18 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:214:40: note: expanded from macro '__deprecated'
19 214 | #define __deprecated __attribute__((__deprecated__))
20    |
21 mk.c:112:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
22 112 |     sem_destroy(&empty);
23    |     ^
24 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
25 53 | int sem_destroy(sem_t *) __deprecated;
26    |
27 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:214:40: note: expanded from macro '__deprecated'
28 214 | #define __deprecated __attribute__((__deprecated__))
29    |
30 mk.c:113:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
31 113 |     sem_destroy(&full);
32    |     ^
33 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
34 53 | int sem_destroy(sem_t *) __deprecated;
35    |
36 /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:214:40: note: expanded from macro '__deprecated'
37 214 | #define __deprecated __attribute__((__deprecated__))
38    |
39 4 warnings generated.
40 screen@killeddy@rees-MacBook-Pro ~ % ./mk
41 Producer-Consumer using POSIX Threads and Counting Semaphores
42 Items produced/consumed: 100000
43
44 Consumer checksum: 5003975013
45
46 Buffer Size : 1
47 Time Taken : 0.113941 seconds
48 Throughput : 886292.71 items/second
49 Consumer checksum: 4992816028
50
51 Buffer Size : 5
52 Time Taken : 0.108567 seconds
53 Throughput : 994668.85 items/second
54 Consumer checksum: 4998674508
55
56 Buffer Size : 10
57 Time Taken : 0.106475 seconds
58 Throughput : 995295.26 items/second
59 Consumer checksum: 5005077408
60
61 Buffer Size : 50
62 Time Taken : 0.899242 seconds
63 Throughput : 1087637.90 items/second
64 Consumer checksum: 5015943308
65
66 Buffer Size : 100
67 Time Taken : 0.115309 seconds
68 Throughput : 867234.99 items/second
69 Consumer checksum: 494374098
70
71 Buffer Size : 1000
72 Time Taken : 0.101758 seconds
73 Throughput : 982721.72 items/second
74 screen@killeddy@rees-MacBook-Pro ~ %
```

2.Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Code:

```
File: jiji.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resourceA;
pthread_mutex_t resourceB;

void *thread1(void *arg)
{
    printf("Thread 1: Trying to acquire Resource A...\n");
    pthread_mutex_lock(&resourceA);
    printf("Thread 1: Acquired Resource A\n");

    sleep(1);

    printf("Thread 1: Trying to acquire Resource B...\n");
    pthread_mutex_lock(&resourceB);
    printf("Thread 1: Acquired Resource B\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resourceB);
    pthread_mutex_unlock(&resourceA);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Trying to acquire Resource B...\n");
    pthread_mutex_lock(&resourceB);
    printf("Thread 2: Acquired Resource B\n");

    sleep(1);

    printf("Thread 2: Trying to acquire Resource A...\n");
    pthread_mutex_lock(&resourceA);
    printf("Thread 2: Acquired Resource A\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resourceA);
    pthread_mutex_unlock(&resourceB);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resourceA, NULL);
    pthread_mutex_init(&resourceB, NULL);

    printf("=== Deadlock Demonstration ===\n\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resourceA);
    pthread_mutex_destroy(&resourceB);

    return 0;
}
```

Output:

```
Screen1:ahillreddy@Screen-MacBook-Pro ~ % nano jiji.c
Screen1:ahillreddy@Screen-MacBook-Pro ~ % gcc jiji.c -o jiji
Screen1:ahillreddy@Screen-MacBook-Pro ~ % ./jiji
=== Deadlock Demonstration ===

Thread 1: Trying to acquire Resource A...
Thread 1: Acquired Resource A
Thread 2: Trying to acquire Resource B...
Thread 2: Acquired Resource B
Thread 1: Trying to acquire Resource B...
Thread 2: Trying to acquire Resource A...

exit
#
```